



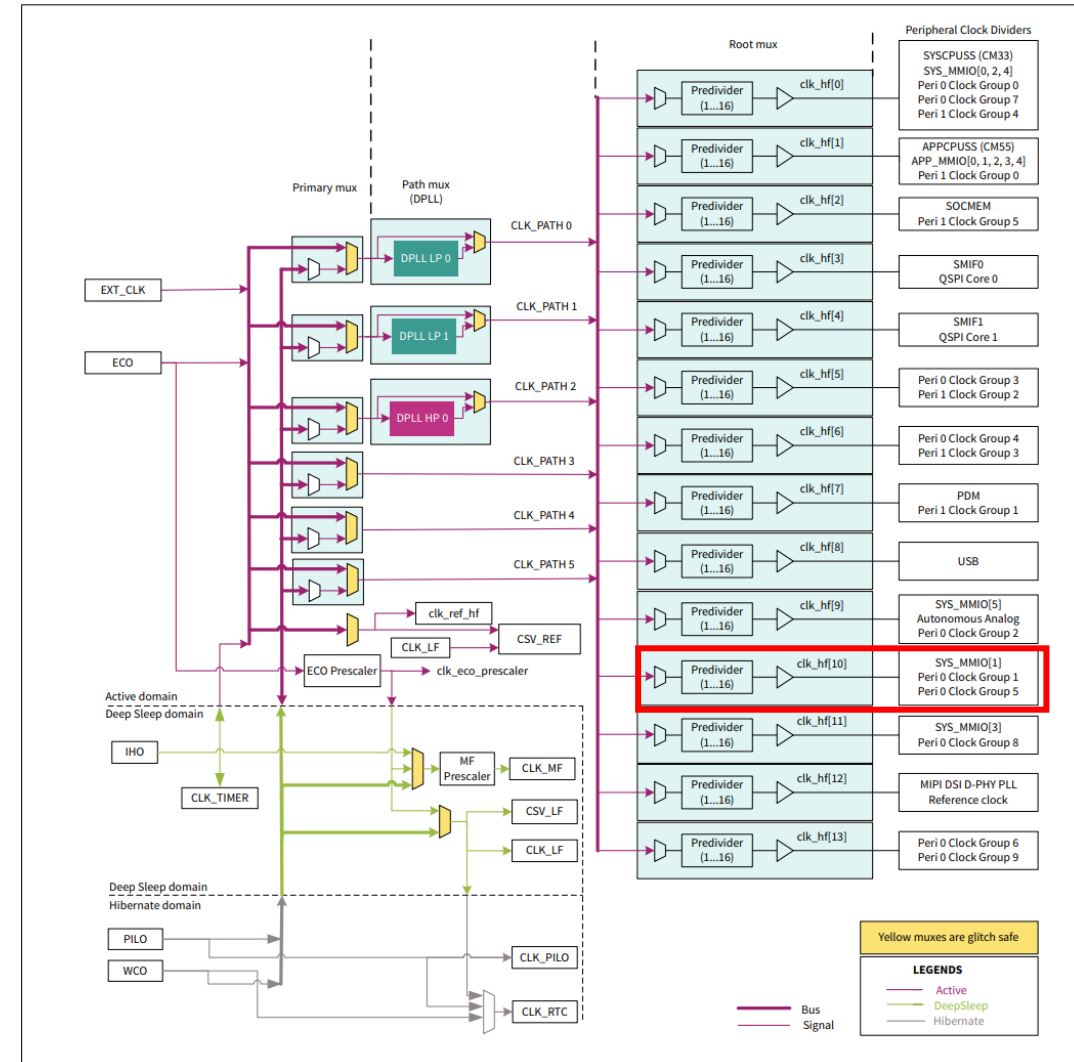
SCB Clock Gating – PSoC Edge E84

Dylan Velasco
21.08.2026



Clock Gating Strategy – Root Clock

- Disable High-Frequency Clock [10].
- Clocked peripherals:
GPIO
SCB
TCPWM



Clock Gating Strategy – Root Clock

- Set the **ENABLE** bit in the **CLK_ROOT_SELECT** register
- Write via Secure Mode

Description: Clock Root Select Register

Address: 0x42401240

Retention: Retained

Bits	7	6	5	4	3	2	1	0
SW Access	Reserved				RW			
HW Access	Reserved				R			
Name	Reserved [7:4]				ROOT_MUX [3:0]			
Bits	15	14	13	12	11	10	9	8
SW Access	Reserved				RW			
HW Access	Reserved				R			
Name	Reserved [15:12]				ROOT_DIV [11:8]			
Bits	23	22	21	20	19	18	17	16
SW Access	Reserved							
HW Access	Reserved							
Name	Reserved [23:16]							
Bits	31	30	29	28	27	26	25	24
SW Access	RW	Reserved						
HW Access	A	Reserved						
Name	ENABLE	Reserved [30:24]						

Bits	Name	Description
31	ENABLE	Enable for this clock root. All clock roots default to disabled (ENABLE==0) except HFCLK0, which cannot be disabled. Default Value: 0

Gating-Free CPU Sleep



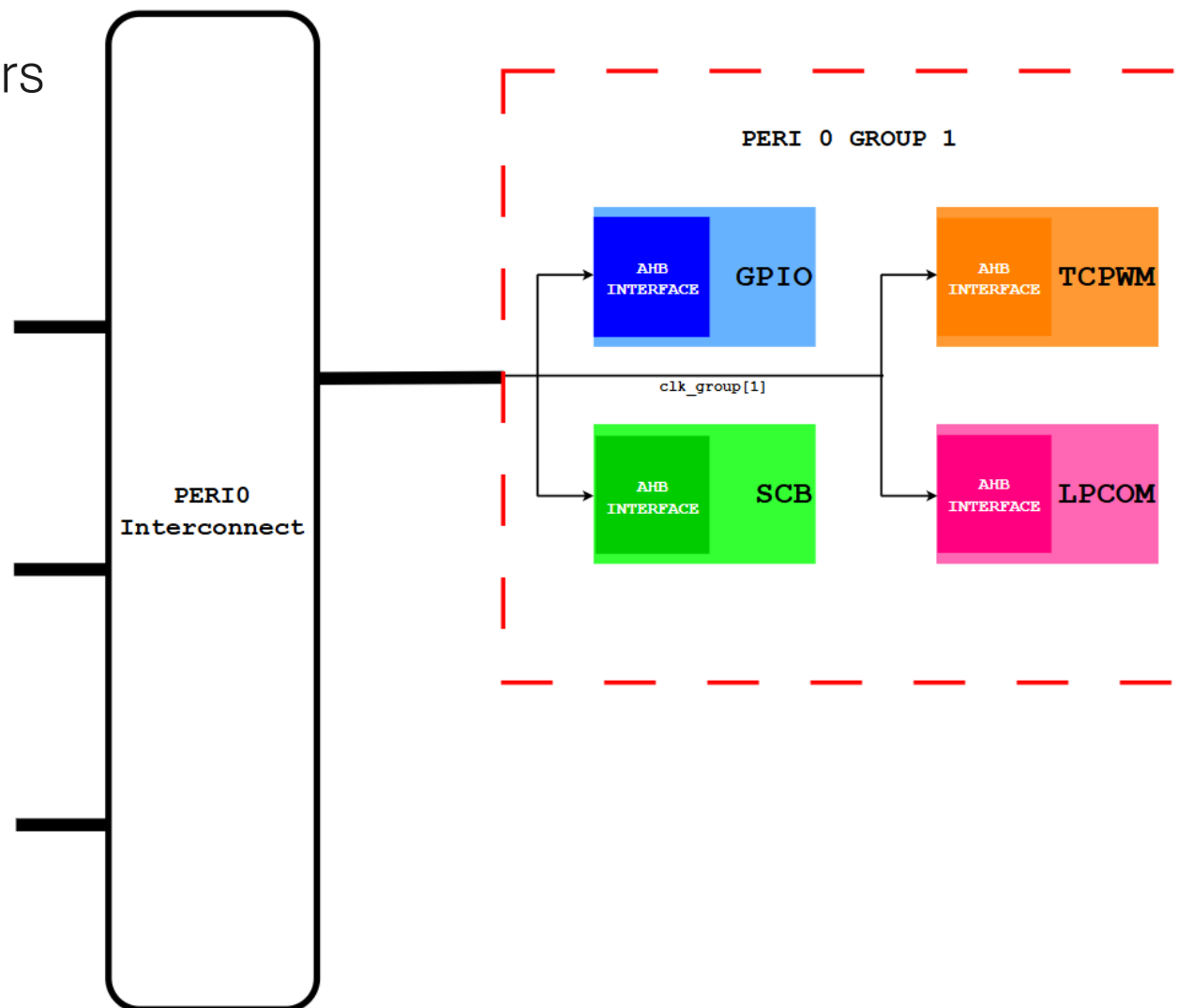
Clock-Gated CPU Sleep



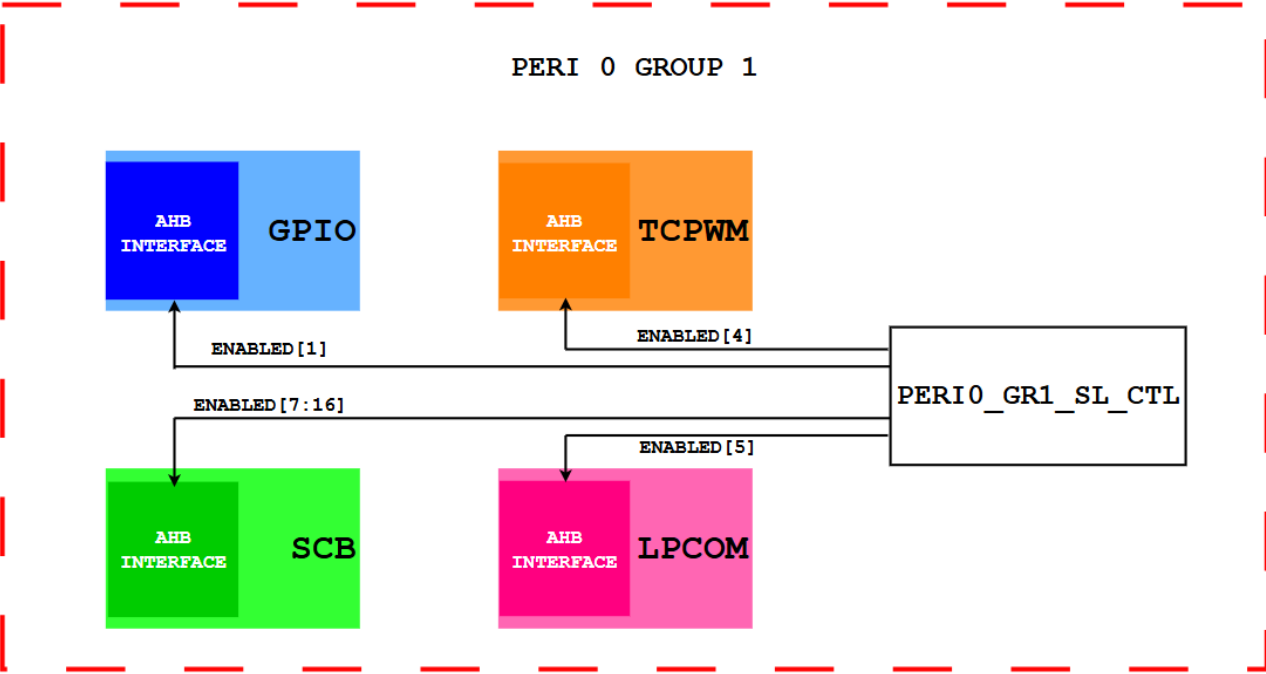
Clock Gating Strategy – Slave Clock

Peri interconnect **connects** AHB masters to **groups** of peripheral devices.

Slave	Slave Number
GPIO	1
TCPWM	4
LPCOMP	5
SCB[2:11]	7 – 16



Clock Gating Strategy – Slave Clock



Each bit indicates whether the respective slave is enabled. If the slave is disabled, its clock is gated off.

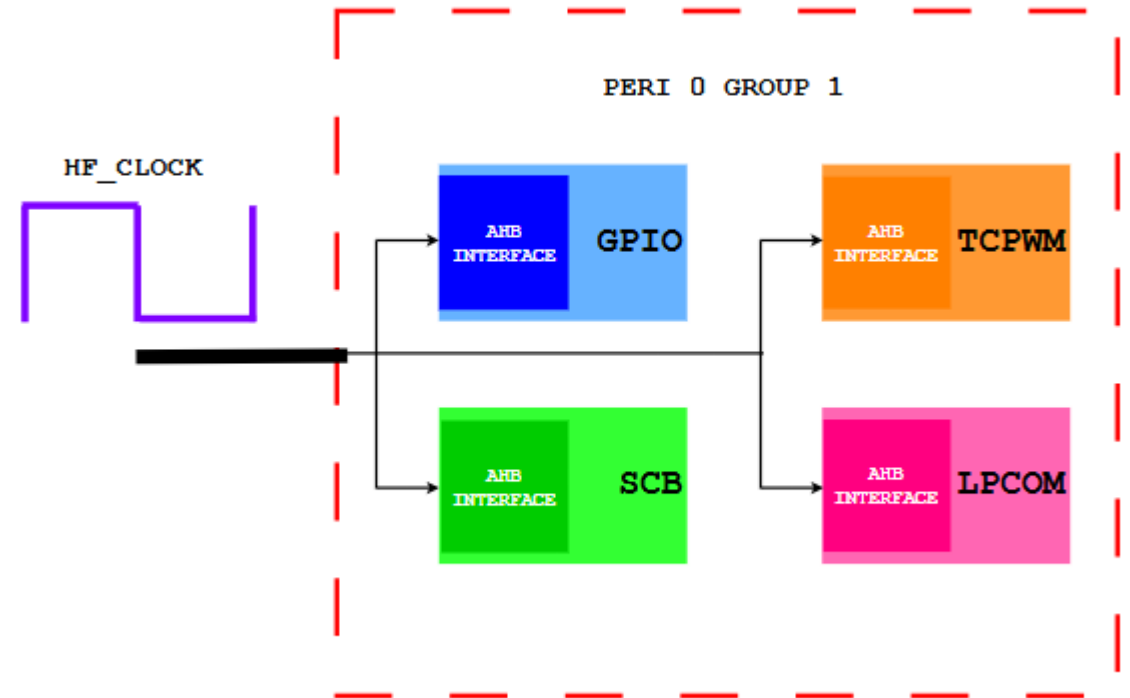
9.1.7 PERIO_GR1_SL_CTL

Description: Slave control
 Address: 0x42004050
 Retention: Retained

Bits	7	6	5	4	3	2	1	0
SW Access	RW							
HW Access	R							
Name	ENABLED [7:0]							
Bits	15	14	13	12	11	10	9	8
SW Access	RW							
HW Access	R							
Name	ENABLED [15:8]							
Bits	23	22	21	20	19	18	17	16
SW Access	Reserved						RW	
HW Access	Reserved						R	
Name	Reserved [23:18]						ENABLED [17:16]	
Bits	31	30	29	28	27	26	25	24
SW Access	Reserved							
HW Access	Reserved							
Name	Reserved [31:24]							

Power Domain - Zephyr

- Groupings of devices powered by a common source called power domain.
- Applications using device A don't need to know about device B to manage their shared power domain.



Power Domain - Devicetree

- In the Device Tree, the power domain is represented as an abstract node.

hf_clock_domain_10:

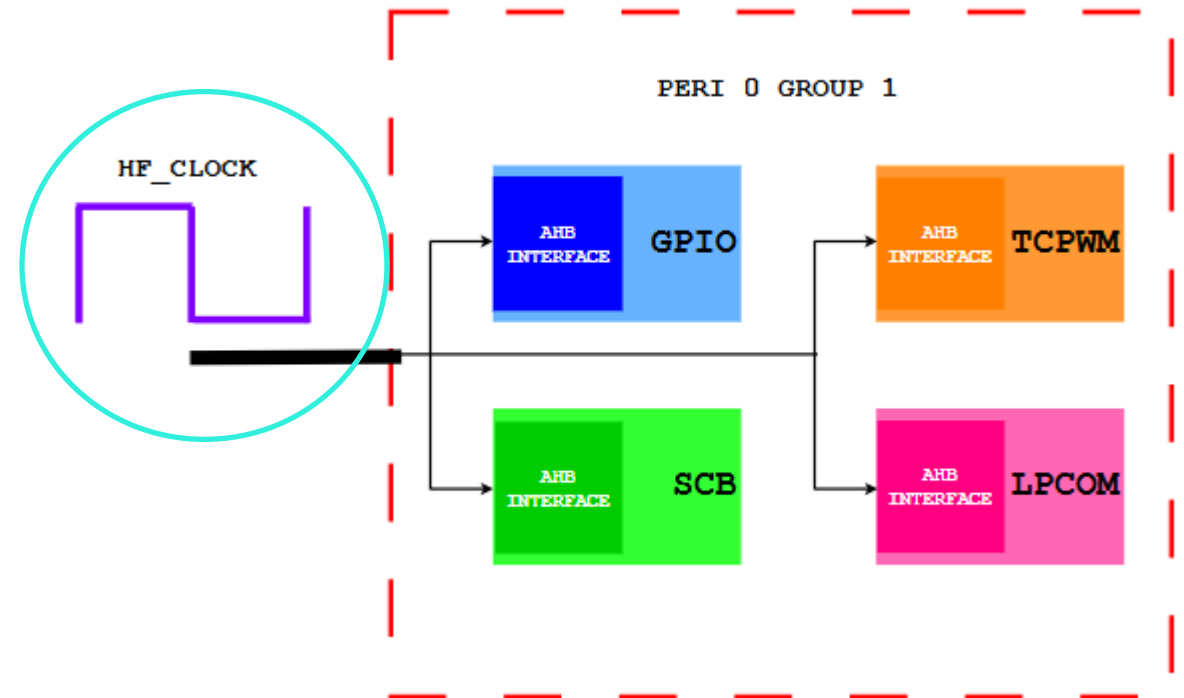
hf_clock_domain_10{

compatible = "power-domain";

#power-domain-cells = <0>;

zephyr,pm-device-runtime-auto;

};

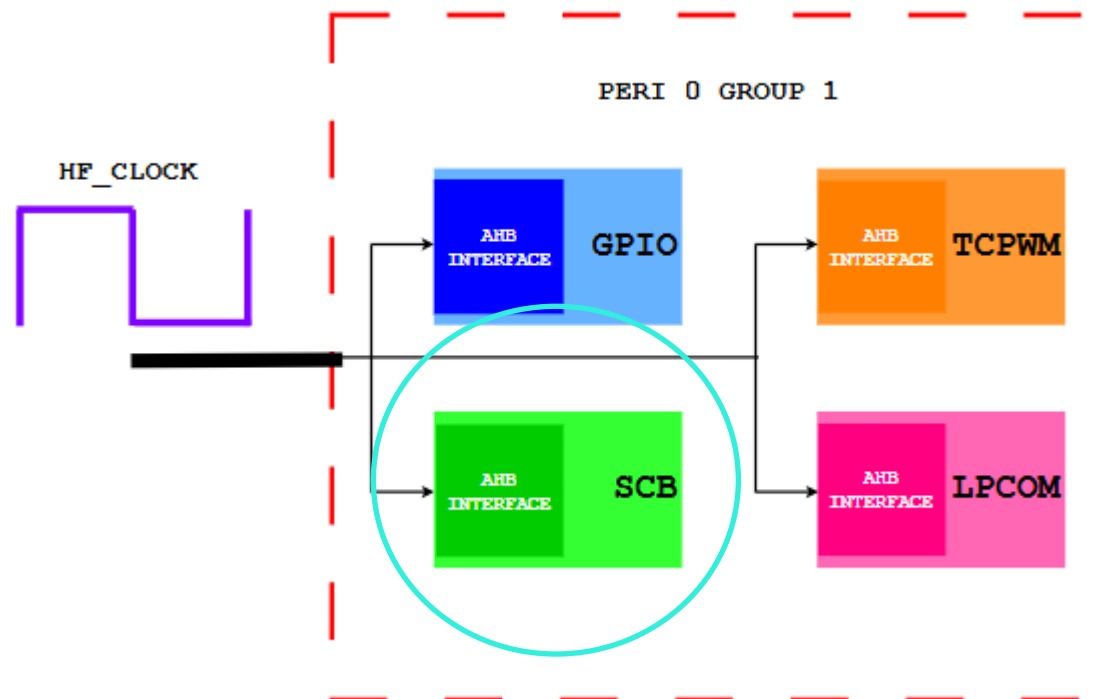


Power Domain - Devicetree

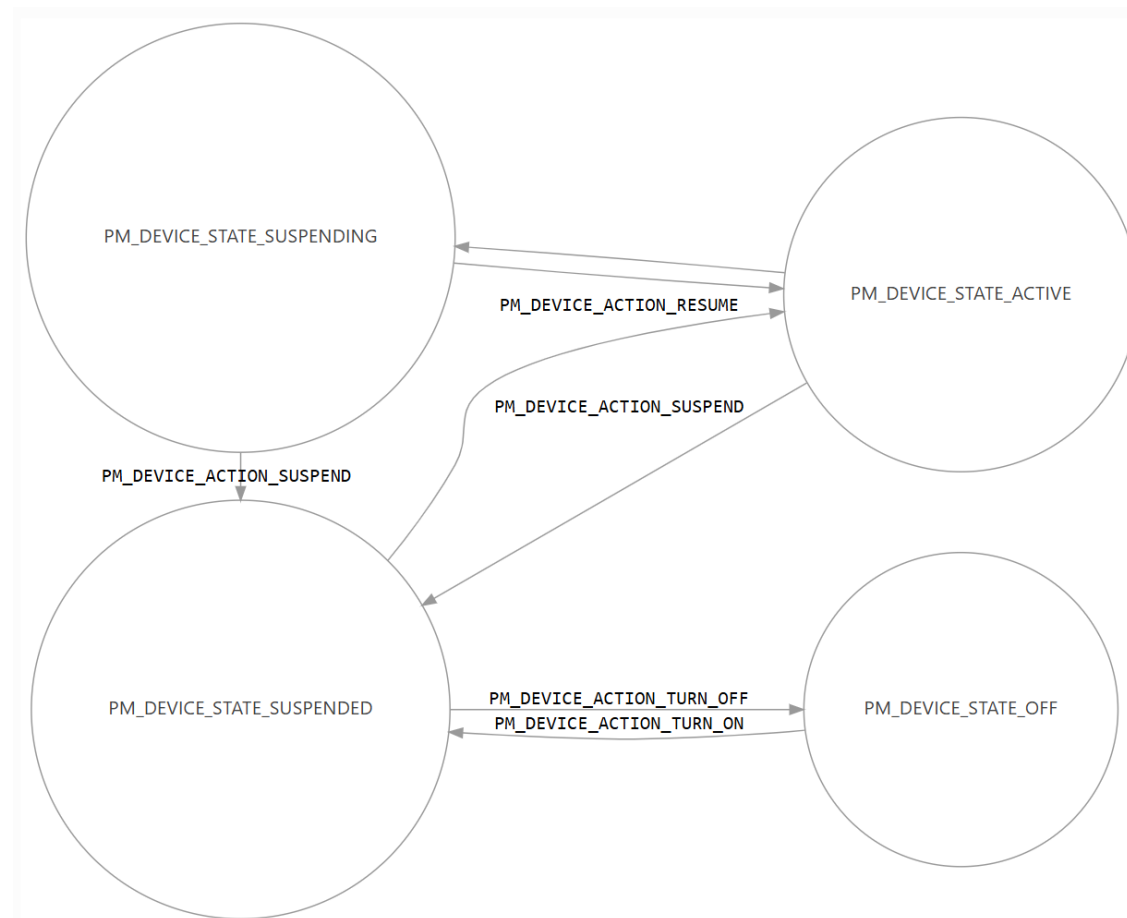
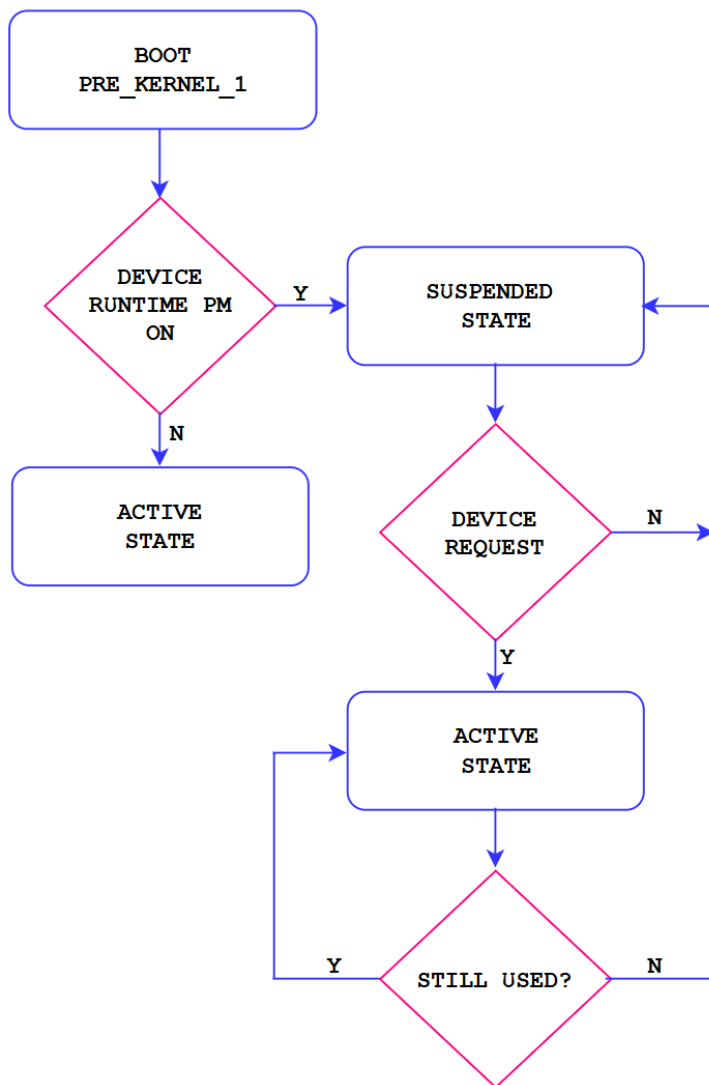
- Each device declares its power domain dependency in the Device Tree.

```
&scb2{
    power-domains = <&hf_clock_domain_10>;
    zephyr,pm-device-runtime-auto;
};
```

zephyr,pm-device-runtime-auto enables device runtime PM in that device.



Device PM – Working Flow

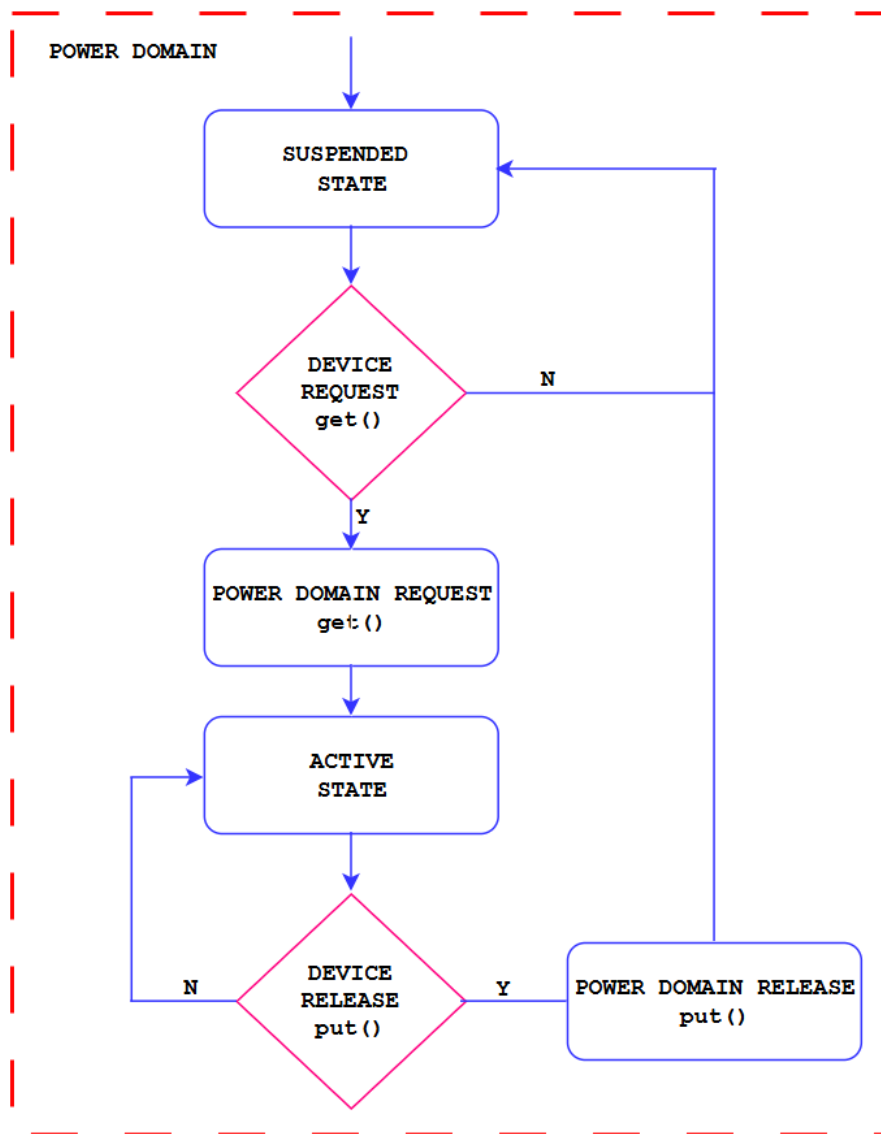


Device PM – Working Flow

- Device drivers indicate when they need the device to be operational and when they don't.
- The PM subsystem tracks usage via reference counting.
- The `get()` and `put()` functions within the driver indicate whether the device is needed, respectively.

Since the power domain is treated as another Runtime PM instance, the same flow and rules apply.

Power Domain – Working Flow



The power domain is automatically requested and released in response to `pm_device_runtime_get()` and `pm_device_runtime_put()` calls on the child devices

Power Domain - Driver

Since the power domain acts as an abstract entity that can be operated on, it requires a dedicated driver:

- Initialization function
- Callback function
- Device PM context
- Zephyr instantiation with pointer to the PM context:

```
static int clock_domain_pd_init(const struct device *dev)
{
    return pm_device_driver_init(dev, clock_domain_pd_action);
}

static int clock_domain_pd_action(*dev, pm_device_action action)
{
    switch (action) {
        case PM_DEVICE_ACTION_RESUME:
            /*resume the domain*/
            pm_device_children_action_run(PM_DEVICE_ACTION_TURN_ON);
        case PM_DEVICE_ACTION_SUSPEND:
            pm_device_children_action_run(PM_DEVICE_ACTION_TURN_OFF);
            /*suspend the domain*/
        case PM_DEVICE_ACTION_TURN_ON:
            /*turn on the domain*/
        case PM_DEVICE_ACTION_TURN_OFF:
            /*turn off the domain*/
    }
}
```

Power Domain - Driver

Since the power domain acts as an abstract entity that can be operated on, it requires a dedicated driver:

- Initialization function
- Callback function
- Device PM context
- Zephyr instantiation with pointer to the PM context:

```
#define INFINEON_CLOCK_PD_INIT(n)
```

```
PM_DEVICE_DT_INST_DEFINE(n, clock_domain_pd_action);  
DEVICE_DT_INST_DEFINE(n, &clock_domain_pd_init,  
                      PM_DEVICE_DT_INST_GET(n),  
                      NULL,  
                      NULL,  
                      PRE_KERNEL_1,  
                      11,  
                      NULL);
```

```
DT_INST_FOREACH_STATUS_OKAY(INFINEON_CLOCK_PD_INIT)
```

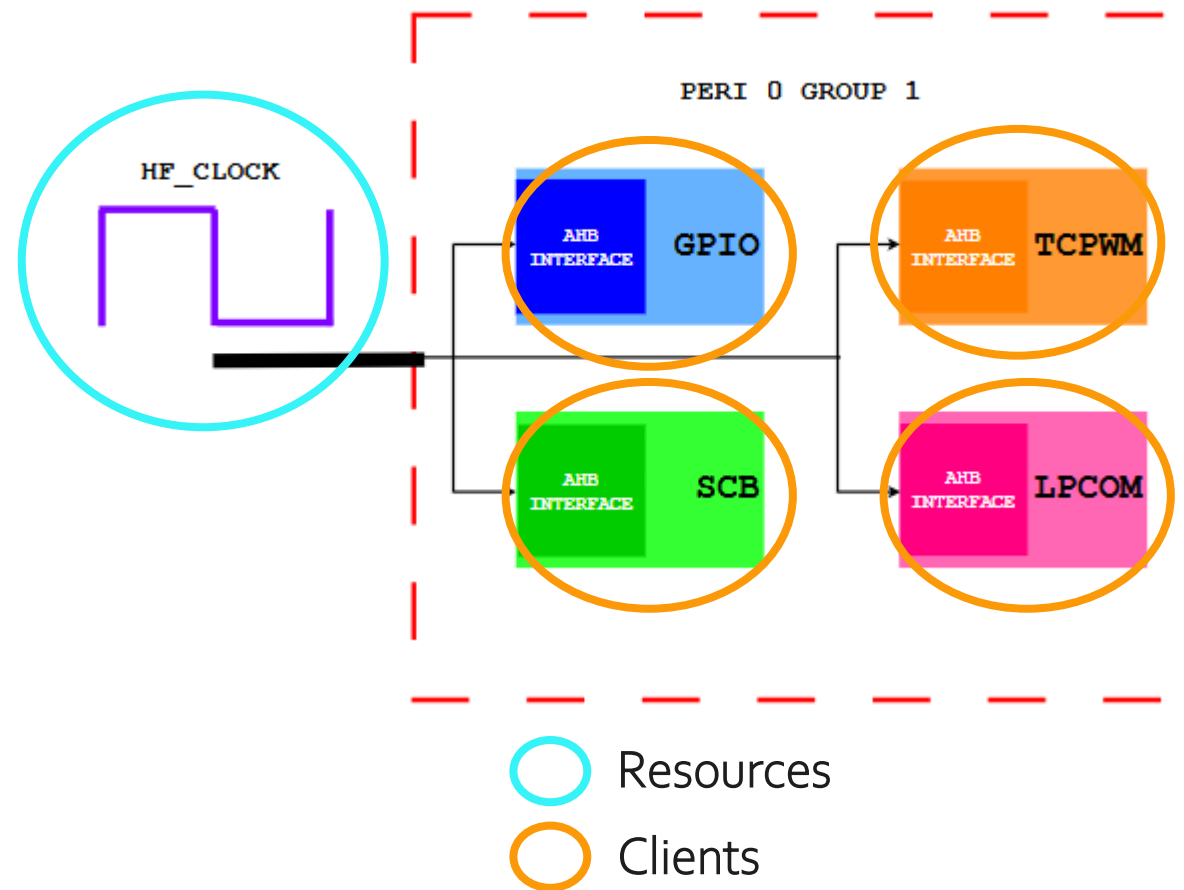
Power Domain - Zephyr



Resource Management – Zephyr

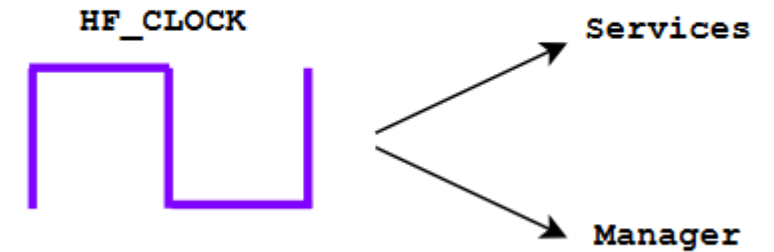
Coordinating a resource among multiple clients (devices, threads) requires a coordination policy.

- Each policy is implemented through a manager.
- Zephyr includes an on/off manager to control shared resource power states (On-Off Manager).



Resource Management – Zephyr

- Manager is embedded into the resource. It's not an external entity.
- The on/off manager tracks how many clients are using the resource's services.
- For hardware resources, management must be handled within the device driver



Resource Management – Zephyr

- Manager instantiation is done via the onoff_manager structure.
- Tracks active client count.
- Two function pointers that define the resource transitions:
 - Count drops to 0 → resource off
 - Count rises above 0 → resource on

```

struct onoff_manager *ifx_clock_get_onoff_manager(const struct device
*dev)
{
    struct fixed_factor_clock_data *data = dev->data;
    return &data->mgr;
}

static void clk_hf_start(struct onoff_manager *mgr, onoff_notify_fn
notify)
{
    /**/
    Cy_SysClk_ClkHfEnable(config->instance);
}

static void clk_hf_stop(struct onoff_manager *mgr, onoff_notify_fn
notify)
{
    /**/
    Cy_SysClk_ClkHfDisable(config->instance);
    notify(mgr, (err == CY_SYSCLOCK_SUCCESS) ? 0 : -EIO);
}

static const struct onoff_transitions clk_hf_transitions =
    ONOFF_TRANSITIONS_INITIALIZER(clk_hf_start, clk_hf_stop,
    NULL);
  
```

Resource Management – Zephyr

- The manager is initialized once per resource instance.

```
static int fixed_factor_clk_init(const struct device *dev)
{
    /**/
    struct fixed_factor_clock_data *const data = dev->data;
    return onoff_manager_init(&data->mgr, &clk_hf_transitions);
}
```

Resource Management – Zephyr

- Once the resource is initialized, clients must request it through the manager before accessing any of its services.
- The client issues a request operation to the manager.
- Client accesses services after manager confirmation.
- Once done, the client sends a release operation to the manager.

```
struct onoff_manager *mgr = ifx_clock_get_onoff_manager(clk_hf10);  
  
struct onoff_client cli;  
  
onoff_request(mgr, &cli);  
  
/*Services*/  
  
onoff_release(mgr);
```

Resource Management – Zephyr



Conclusions

- The power domain mechanism automatically calls get() and put() on the domain when a child device is used.
- The power domain is an external entity linked to devices through the Device Tree.
- By implementing Runtime PM in the power domain's child devices, the power domain is used implicitly and the drivers require no additional changes.
- The resource management mechanism requires an explicit call to the manager for each resource.
- The logical connection between the resource and the client is established in code.
- It is less portable.

